Twin inputs for the shared parts

The Twin team's contribution to **Part 0** (triplet overview and integration) and **Part 4** (Test Book). The Guardian and Brain columns state what each partner layer is designed to do, from its code; what we actually observed in the joint runs is in the last section, *Observed in the joint runs.*

Part 0 input: one scenario traced through the three layers

Port flood in La Goulette (scenario T3). A storm triples the inflow to the drainage channel in zone 2. The Twin's physics raises the real water level at floats `z2-wl1` and `z2-wl2`, and both keep reporting ten times per second on Kafka, each reading signed with its own Ed25519 key (for example `"device_id":"z2-wl1"`, `"value":6.92`, `"unit":"meters"`, `"status":"ok"`, plus its signature). The Guardian verifies every signature with the public key it fetched from the Twin over MCP (`get_device_key`), checks the reading against its rules and models, and passes it on as trusted. The Brain sees the level climbing toward the 8 m danger line on two agreeing, trusted sensors and calls the Twin's MCP tool `set_valve {"device_id":"z2-wl1","target_state":"open"}`. The Twin applies the command to the physics: the drain opens, the level peaks around 7.3 m and never crosses 8 m. The ground-truth ledger records the storm, the command (caller `brain`, executed) and the real level, so the run log `logs/t3-*.log` proves the outcome independently of what any layer claims.

What the Twin contributes to the triplet: the stream (Kafka, signed, $\leq$ 300 B, keyed by device), the command surface (16 MCP tools: queries, actuation, containment), the attack engine that gives the Guardian something real to catch, and the ground truth the jury can check afterwards.

Shared decisions the Twin relies on: Kafka with SASL on the Guardian's three-broker cluster (the Twin writes `telemetry-ingest` as user `twin`); MCP over HTTP on port 8000 with one bearer token per layer; Ed25519 over the sorted compact JSON without `sig`; device public keys come only from `get_device_key`; containment (isolate, quarantine) is executed by the Twin at the Guardian's request.

Part 4 input: Twin scenarios

Every scenario starts with one command and writes `logs/tN-<timestamp>.log`:

- `export RED_TEAM_TOKEN=$(docker exec twin_layer cat /app/data/red_team.token)`
- `python3 scenarios/run_all.py --with-mocks` (all 11, about 5 minutes at `SIM_SPEED=20`), or one at a time, e.g. `python3 scenarios/t6_fake_port_flood.py --with-mocks`

`--with-mocks` uses our stand-in Guardian and Brain; `--partners` runs the same trigger against the real partner layers and sends no actuation or containment itself.

Coverage of the interactions required by the rules. Twin → Brain (real events): T2 accident, T3 port flood, T4 fire, T5 grid overload. Twin → Guardian (healthy vs unhealthy data): T1 baseline, T7 stuck sensor, T8 rogue device, T9 stolen key, T10 link tampering. Brain ↔ Guardian (trust changes decisions): T6 fake port flood, T9 stolen key. All three: T11 storm with a lying actuator.

Table 1: Twin scenarios with concrete, observable expected outputs

ID	Name	Expected: Twin	Expected: Guardian	Expected: Brain	How to verify
T1	Calm baseline	42/42 devices, every frame $\leq$ 300 B, all signatures valid, about 420 msg/s, no containment	Accepts all readings, no flags	No actuation	checks `devices_42_emitted`, `all_signatures_verified`, `message_rate_nominal`
T2	Traffic accident, zone 1	`z1-tc1` flow drops to $\times$0.3, zone calls +6/min; resolved only by a matching dispatch	Readings trusted	`dispatch` medical in zone-1	journal: dispatch executed, `resolved_by: dispatch:*`
T3	Port flood, zone 2	Storm $\times$3; after `set_valve open` the peak stays below 8 m (observed 7.25 m)	Readings trusted (unit `"meters"` accepted)	`set_valve z2-wl1 open`	`mitigated_storm_level_under_8m`
T4	Fire, zone 5	AQI at `z5-aq1` above 150, calls +8/min; resolved by a fire dispatch	Readings trusted	`dispatch` fire in zone-5	journal: `resolved_by: dispatch:*`
T5	Grid overload, zone 3	Load $\geq$ 118 MW; blackout above 114 MW for 60 s; resolved by shedding	Readings trusted	`switch_grid z3-pl1 shed`	journal: `resolved_by: actuation:switch_grid`
T6	Fake port flood	`z2-wl1` broadcasts 9.5 m with a **valid** signature; true level and `z2-wl2` stay below 5 m	Flags the two floats disagreeing; `isolate_device z2-wl1`	Does **not** open the valve	`z2_wl1_isolated`, `no_set_valve_command_emitted`
T7	Stuck sensor	`z4-aq1` frozen at 42.0 while the real AQI moves	Detects the stuck signal; isolates `z4-aq1`	Ignores the device	`z4_aq1_isolated_for_stuck_fault`
T8	Rogue device	Frames from unregistered `rg-p1`; `get_device_key` returns `null`; blocked after quarantine	Rejects as unknown device; `quarantine_device rg-p1`	Never acts on it	`mcp_device_key_null_for_rogue`, `rogue_device_quarantined`
T9	Stolen key	`z1-pl1` sends 180 MW (above the 120 MW maximum) with a valid signature	Flags the impossible value; `quarantine_device z1-pl1`	No `switch_grid` on that device	`z1_pl1_quarantined`, `no_switch_grid_dispatched`
T10	Link tampering	On `z3-tc1`: corrupted bytes, broken signature, replayed `reading_id`, 5 s delay, dropped frames	Bad signature, replay and late-frame detection, silence	Ignores rejected data	five sub-checks in `logs/t10-*.log`
T11	Storm + lying actuator	`set_valve open` answers "ok" but the ledger records `executed: false` and the level keeps rising (+1.4 m in 5 min)	—	Notices the level not falling and escalates	`journal_records_executed_false`, `physical_water_level_rise_ge_0_3m`

One scenario in the rule format (T6, fake flood at the port). *Layers:* Twin, Guardian, City Brain. *Description:*

float `z2-wl1` reports 9.5 m with a valid signature while the real level, and the second float `z2-wl2`, stay near 3.5 m. *Trigger:* `python3 scenarios/t6_fake_port_flood.py` (injects `device_physical` on `z2-wl1`, value 9.5). *Expected, Twin:* streams the fake 9.5 m with a valid signature, keeps the true level in the ledger, executes `isolate_device` when asked. *Expected, Guardian:* marks `z2-wl1` untrusted (it disagrees with `z2-wl2` by more than 5 m) and calls `isolate_device z2-wl1`. *Expected, Brain:* does not call `set_valve` for zone 2 and records why. *How to verify:* timeline screen (fake reading, Guardian isolation, no Brain action); log `logs/t6-*.log` shows `spoofed_reading_signature_verified`, `z2_wl1_isolated` and `no_set_valve_command_emitted` as PASS.

Observed in the joint runs

Two full runs on the Guardian's three-broker SASL cluster, with all three layers on one laptop.

Table 2: What the joint runs actually showed

	Run 1 (8 Nov, morning)	**Run 2 (8 Nov, evening, Guardian's final version)**
Twin delivery to `telemetry-ingest`	779,325 of 779,360 (19 failures at broker startup)	126,344 of 126,378 (14 failures at broker startup)
Twin-side scenario checks	All pass in all 11 scenarios	Not run (see last row)
Guardian	No output: its validation job read the wrong topics and had no SASL settings	Metadata producer listed all 42 devices over MCP; the validation job processed the stream; the AI orchestrator ran about 60,000 readings behind and labelled healthy devices `sensor_is_down`; its containment calls timed out (3 s limit)
Brain	Rejected 690 bad signatures (T10) and 352 rogue frames (T8) itself; T3 opened valves (6 commands, peak 6.62 m); T6 correctly held back; no incident opened for T2, T4, T5	Stopped after its first decisions: not allowed to write `brain.events` on the new cluster, and the Guardian publishes verdicts to `telemetry-confirmation`, not the `guardian.verdicts` it reads
Scenarios passed end to end	3 of 11 (T1, T3, T11)	Not run: the CPU was saturated (Twin at 22–68 readings/s, MCP replies 10–25 s), so the results would have measured the laptop, not the layers

What this shows. The Twin side of every interface works against the real partner infrastructure. The remaining gaps are in the Guardian–Brain link (topic names, ACLs, a verdict signing key that is never published) and in running the whole stack on one machine. For the demo, each layer runs on its own machine.